DTI201 • ฉบับเรื่องเล่าร้านกาแฟ

ย้ายร้านไปที่ใหม่ โดยเตรียมของให้พร้อม

จากรถบ้านร้านกาแฟสู่ Container, Docker และ Book API



Container → Docker → Docker Compose

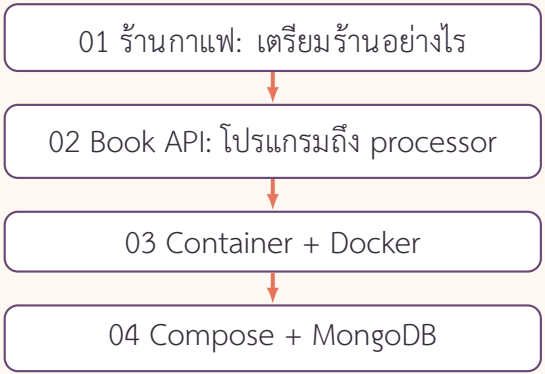
เมื่อจบคาบ เราจะอธิบายการรัน Book API ได้

เริ่มจาก **ร้านกาแฟ** เพื่อเห็นปัญหาการเตรียมสถานที่และอุปกรณ์

ตาม **Book API** จาก JavaScript และ Node.js ลงไปถึง kernel และ processor

เปรียบเทียบ **image กับ container**, ทดลองคำสั่ง Docker แล้วอ่านแผนบริการของ Docker Compose

สุดท้ายอธิบาย port และที่เก็บข้อมูลที่ MongoDB ใช้ใน Lab 10 ด้วยหลักฐานจากใบงาน



ลองคิดหรือทดลอง: เมื่อจบคาบ เราควรชี้ให้ได้ว่า MongoDB เก็บข้อมูลที่ใด

ร้านเดียวกันเปิดได้หลายสถานที่

สมมติเราต้องเปิดร้านกาแฟในสามสถานที่: บ้านที่จัดเป็นร้าน, จุดกางเต็นท์ และรถบ้านที่ดัดแปลงเป็นร้านกาแฟ

ไม่ว่าจะอยู่ที่ใด เรายังต้องมีเครื่องชง แก้ว น้ำ ไฟ และคนชง. แต่แต่ละสถานที่เตรียมของและย้ายร้านต่างกัน

เราจะใช้เรื่องนี้ช่วยคิดว่าโปรแกรมต้องเตรียมอะไรบ้างก่อนรันบนเครื่องใหม่



ลองคิดหรือทดลอง: ถ้าพรุ่งนี้ย้ายร้านไปสนามใหม่ สิ่งใดต้องเตรียมอีกครั้ง?

การจัดร้านใหม่ทุกครั้งใช้เวลาและอาจตกหล่น

เมื่อไปกางเต็นท์เปิดร้าน เราต้องนำโต๊ะ เครื่องชง ปลั๊ก สายไฟ และน้ำไปติดตั้งทีละอย่าง

ถ้าลืมสายไฟ เครื่องชงอาจพร้อมแต่เปิดร้านไม่ได้. คนละทีมอาจจัดอุปกรณ์ไม่เหมือนกัน แม้จะขายกาแฟเมนูเดียวกัน

ในงานซอฟต์แวร์ Book API ก็ต้องมี Node.js และแพ็กเกจที่ตรงกับโปรแกรม



ลองคิดหรือทดลอง: ถ้าเพื่อนมีไฟล์ Book API แต่ยังไม่ได้ติดตั้ง Node.js จะเกิดอะไรขึ้น?

รถบ้านช่วยจัดอุปกรณ์ร้านไว้เป็นชุด

รถบ้านร้านกาแฟเตรียมเคาน์เตอร์ เครื่องชง และอุปกรณ์ไว้ด้วยกัน จึงเคลื่อนย้ายชุดอุปกรณ์ไปเปิดร้านในที่ใหม่ได้สะดวกขึ้น

แต่รถยังต้องอาศัยถนน จุดจอด ไฟ และน้ำที่ใช้ร่วมกับสถานที่. การเตรียมรถไม่ทำให้ทุกสถานที่พร้อมโดยอัตโนมัติ

ภาพนี้ช่วยเริ่มคิดเรื่องการนำโปรแกรมและสิ่งที่ต้องใช้ไปยังเครื่องอื่น



ลองคิดหรือทดลอง: อะไรอยู่ในรถ และอะไรยังต้องเตรียมจากสถานที่ใหม่?

ร้านหนังสือใช้แนวคิดการเตรียมชุดเดียวกัน

เปลี่ยนจากรถขายกาแฟเป็นรถร้านหนังสือ: มีชั้นหนังสือ เครื่องคอมพิวเตอร์ และที่เก็บหนังสือที่ต้องใช้ต่อ

ในโปรเจกต์ของเรา หน้า React แสดงรายการหนังสือ; Book API รับคำขอ; MongoDB เก็บข้อมูลหนังสือ

เราจะใช้ร้านกาแฟช่วยจินตนาการ แล้วกลับมาชี้ส่วนจริงของ Book API และ MongoDB ทุกครั้ง



ลองคิดหรือทดลอง: ถ้าหน้าร้านหนังสือแสดงรายการได้ โปรแกรมใดรับคำขอ และโปรแกรมใดเก็บข้อมูล?

ช่วงที่ 1 / 4

โปรแกรมต้องอาศัยหลายชั้นจึงทำงานได้

ไล่จาก Book API ลงไปถึงตัวประมวลผล

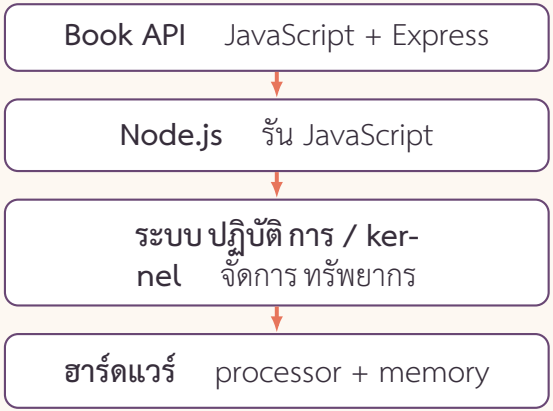
ร้านกาแฟ → Book API → Container → Docker Compose

Book API ไม่ได้ทำงานลำพัง

แอป Book API เป็นโค้ด JavaScript ที่เราสร้างเพื่อรับคำขอและตอบข้อมูลหนังสือ

Node.js อ่านและรัน JavaScript; Express เป็นแพ็กเกจที่ช่วยเขียนเว็บเซิร์ฟเวอร์. แอปขอใช้ไฟล์ เครือข่าย และหน่วยความจำผ่านระบบปฏิบัติการ

ด้านล่างสุด CPU หรือ processor ทำงานตามคำสั่งของเครื่อง โดยใช้หน่วยความจำและอุปกรณ์ร่วมด้วย



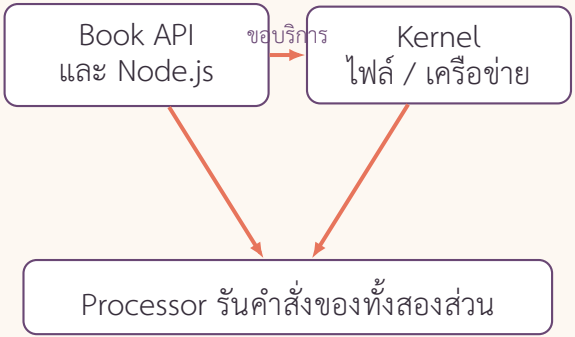
ลองคิดหรือทดลอง: ถ้าไม่มี Node.js เครื่องจะรัน Book API JavaScript ของเราได้อย่างไร?

Processor รันทั้งโค้ดโปรแกรมและโค้ด kernel

Node.js เป็นกระบวนการของโปรแกรม. Processor รันคำสั่งของ Node.js และ Book API โดยตรงตามการทำงานของเครื่อง

เมื่อโปรแกรมต้องการใช้บริการของระบบ เช่น เปิดไฟล์หรือรับข้อมูลเครือข่าย โปรแกรมเรียกใช้ kernel; processor จึงรันโค้ด kernel เพื่อจัดการงานนั้นด้วย

ภาพ stack ช่วยจัดหน้าที่ แต่ไม่ได้หมายความว่าโค้ดทุกบรรทัดต้องผ่าน kernel ก่อนถึง processor



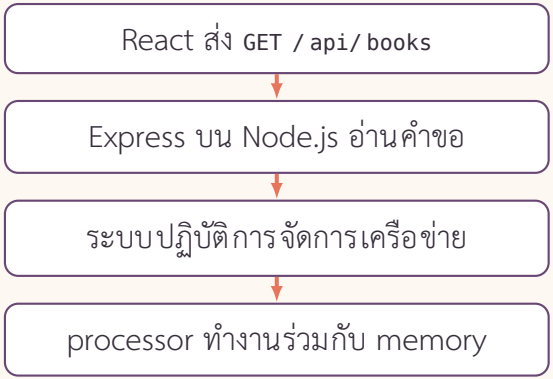
ลองคิดหรือทดลอง: เมื่อ API ขออ่านไฟล์ ส่วนใดช่วยจัดการไฟล์ และส่วนใดรันคำสั่ง?

เมื่อกดปุ่ม ข้อมูลเดินผ่านแต่ละชั้น

ผู้ใช้กดปุ่มใน React แล้วเบราว์เซอร์ส่งคำขอ HTTP ไปที่ Book API

Node.js รันโค้ด Express; โค้ดขอให้ระบบปฏิบัติการรับส่งข้อมูลผ่านเครือข่าย. Processor ทำงานตามคำสั่งของโปรแกรมและระบบ

คำตอบย้อนกลับผ่านเครือข่ายไปให้ React แสดงบนหน้าจอ. แต่ละชั้นมีหน้าที่ต่างกัน



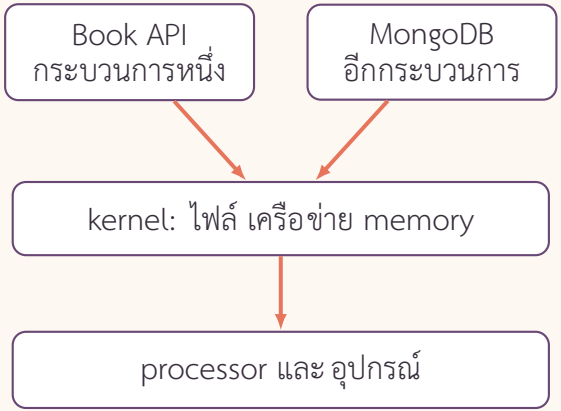
ลองคิดหรือทดลอง: วงชั้นที่รับ HTTP และชั้นที่รันโค้ด JavaScript

Kernel จัดการทรัพยากรให้โปรแกรมใช้

Kernel เป็นแกนของระบบปฏิบัติการ. โปรแกรมขอให้ kernel ช่วยจัดการงาน เช่น เปิดไฟล์ รับส่งข้อมูลเครือข่าย และใช้หน่วยความจำ

โปรแกรมหลายตัวใช้ kernel เดียวกันได้ โดยระบบจัดการสิ่งที่แต่ละกระบวนการมองเห็นและใช้ได้

ชั้นนี้สำคัญเพราะ Linux containers ที่รันบนระบบเดียวกันใช้ Linux kernel ร่วมกัน



ลองคิดหรือทดลอง: Book API กับ MongoDB ต้องมี CPU แยกคนละตัวเสมอหรือไม่?

แนวคิดการแยกโปรแกรมเกิดก่อน Docker

ปี 2000 ระบบ FreeBSD มี **jail** เพื่อแยกพื้นที่ทำงานของผู้ใช้และบริการบนระบบเดียวกัน

ต่อมา Linux เพิ่มความสามารถแยกมุมมองและจำกัดทรัพยากร; โครงการ LXC รวมเครื่องมือสำหรับ Linux containers ราวปี 2008

ปี 2013 Docker ช่วยให้การสร้าง แจกจ่าย และเปิด container images เป็นขั้นตอนที่ทีมพัฒนานำไปใช้ร่วมกันได้ง่ายขึ้น

อ่านประวัติ Linux containers อ่านที่มา Docker



การแยกโปรแกรม → เครื่องมือสำหรับสร้างและแจกจ่าย

ลองคิดหรือทดลอง: Docker สร้างแนวคิดการแยกโปรแกรมขึ้นมาเป็นครั้งแรกหรือไม่?

เครื่องใหม่อาจมีชั้นซอฟต์แวร์ไม่เหมือนกัน

เครื่องหนึ่งมี Node.js รุ่นที่โปรเจกต์ต้องใช้ แต่อีกเครื่องอาจมีอีกรุ่นหรือยังไม่มี Node.js

ถ้าเราคัดลอกเฉพาะ `app.js` แล้วลืมแพ็กเกจ Express โปรแกรมอาจเริ่มไม่ได้ แม้ไฟล์โค้ดจะเหมือนกัน

ปัญหาเหมือนนำเมนูร้านกาแฟไปถึงแคมป์ แต่เครื่องชงและสายไฟยังไม่ได้เตรียม



ลองคิดหรือทดลอง: นอกจากไฟล์โค้ด Book API ต้องเตรียมอะไรบนเครื่องใหม่?

เครื่องเสมือนมี kernel ของตัวเอง

Virtual machine (VM) จำลองเครื่องให้ระบบปฏิบัติการอีกชุดทำงาน. แต่ละ VM มี guest operating system และ kernel ของตน

ภาพบ้านช่วยนึกถึงพื้นที่ที่มีระบบภายในของตนเอง แต่คอมพิวเตอร์จริงใช้ hypervisor แบ่งทรัพยากรให้ VM

VM เหมาะกับหลายงาน โดยเฉพาะเมื่อจำเป็นต้องมีระบบปฏิบัติการหรือ kernel แยก



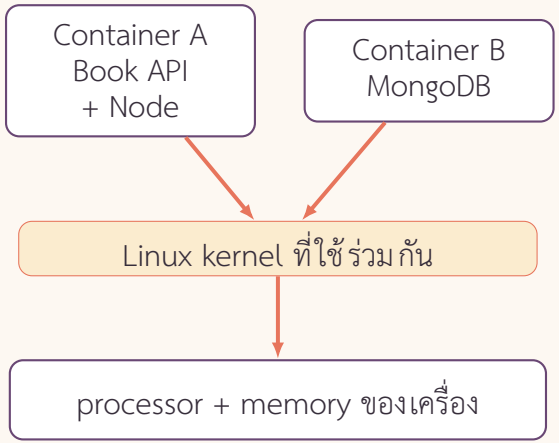
ลองคิดหรือทดลอง: ถ้า VM สองตัวใช้ระบบปฏิบัติการต่างกัน แต่ละตัวมี kernel ของใคร?

Container ใช้ kernel ร่วมแต่แยกมุมมองของโปรแกรม

Linux container เป็นกระบวนการที่รันโดยมีมุมมองไฟล์และเครือข่ายแยกตามการตั้งค่า. หลาย container บน Linux host เดียวกันใช้ kernel เดียวกัน

Image เตรียมแอป runtime และไฟล์ที่ต้องใช้; container เป็น instance ที่สร้างจาก image นั้น

บน Windows หรือ macOS Docker Desktop มักรัน Linux containers ภายใน Linux VM ที่จัดการให้



ลองคิดหรือทดลอง: ชี้ Node.js ที่อยู่กับ Book API และชี้ kernel ที่ containers ใช้ร่วมกัน

ช่วงที่ 2 / 4

รถบ้านช่วยให้เห็นสิ่งที่เตรียมและสิ่งที่ใช้ร่วม

เปรียบเทียบ แล้วกลับมาดูระบบจริง

ร้านกาแฟ → Book API → Container → Docker Compose

ชุดร้านที่เตรียมไว้คล้าย image

ถ้าทีมออกแบบรถบ้านร้านกาแฟไว้หนึ่งแบบ พร้อมรายการเครื่องชง โต๊ะ และอุปกรณ์ ทีมสามารถเตรียมรถหลายคันตามแบบเดียวกัน

ในคอมพิวเตอร์ **image** เป็นชุดไฟล์ตั้งต้นสำหรับสร้าง container. อาจมี Book API, Node.js และแพ็กเกจที่แอปต้องใช้

เมื่อสร้าง container จาก image เราได้ instance ที่เปิด หยุด หรือลบได้



ลองคิดหรือทดลอง: แบบรถกับรถคันที่จอดขายกาแฟอยู่ ต่างกันเหมือน image กับ container อย่างไร?

รถบ้านไม่ได้พกทั้งแคมป์ติดตัว

รถบ้านมีอุปกรณ์ร้าน แต่ยังใช้พื้นที่จอดและสาธารณูปโภคของสถานที่

เปรียบกับ Linux container: image มีไฟล์แอปและส่วนประกอบที่ต้องใช้ แต่ container ใช้ kernel ของระบบที่รันมัน

ขอบเขตของภาพเปรียบเทียบ: รถจริงมีล้อและเครื่องยนต์ของตนเอง; container ไม่ได้มี CPU หรือ kernel ส่วนตัวติดมาด้วย



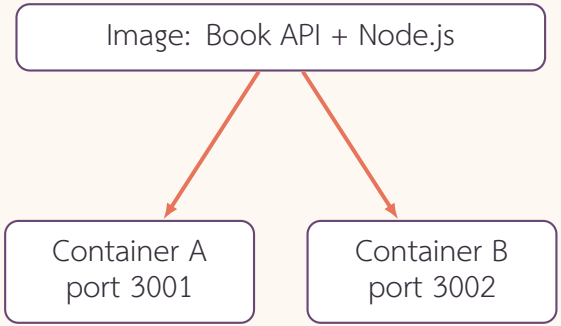
ลองคิดหรือทดลอง: ส่วนใดของคอมพิวเตอร์ทำหน้าที่ร่วมกันระหว่าง containers?

Image หนึ่งชุดสร้าง container ได้หลายตัว

เหมือนเราเตรียมรถร้านกาแฟหลายคันตามแบบเดียวกัน เราสามารถสร้าง Book API containers สองตัวจาก image เดียวกัน

แต่ละตัวมีสถานะการรันและชั้นข้อมูลที่เขียนเพิ่มของตน. หยุดตัวหนึ่งไม่ได้แปลว่าอีกตัวต้องหยุด

หากเปิดให้เข้าผ่านเครื่องเดียวกัน ต้องกำหนด port ฝั่งเครื่องไม่ให้ชนกัน



ลองคิดหรือทดลอง: ถ้าหยุด container A image และ container B ยังอยู่หรือไม่?

Volume ช่วยเก็บข้อมูลเมื่อเปลี่ยน container

สมมติรถร้านกาแฟมีสมุดรายการขาย. ถ้าสมุดอยู่ในรถที่นำออกไป เราอาจหาไม่พบเมื่อเปลี่ยนรถ

ถ้าเก็บสมุดไว้ในตู้เก็บที่แยกจากรถ รถคันใหม่สามารถนำสมุดเดิมมาใช้ได้

ใน Lab 10 MongoDB เขียนไฟล์ฐานข้อมูลลง volume. การหยุดหรือลบ container ไม่จำเป็นต้องลบ volume นั้น



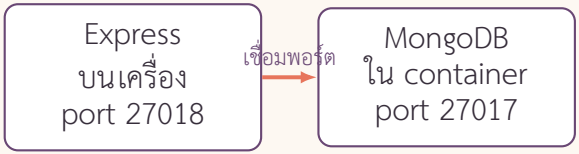
ลองคิดหรือทดลอง: ถ้าต้องการให้หนังสือยังอยู่หลังสร้าง MongoDB container ใหม่ ควรเก็บข้อมูลที่ใด?

ทางเข้าร้านคล้าย port ที่แอปใช้ติดต่อ

รถบ้านมีหน้าต่างรับคำสั่งซื้อ. ลูกค้าต้องรู้ว่าควรติดต่อช่องใด

โปรแกรมใช้ port แยกช่องทางเครือข่าย. ใน Lab 10 เครื่องของเราใช้ port `27018` ไปยัง port `27017` ใน MongoDB container

ตัวเลขสองฝั่งอาจต่างกัน. เราต้องอ่านทิศทางจากเครื่องไปยัง container ให้ถูก




```
127.0.0.1:27018:27017
```

ลองคิดหรือทดลอง: ถ้า Express อยู่บนเครื่อง ต้องติดต่อ port 27018 หรือ 27017?

ช่วงที่ 3 / 4

Docker ช่วยสร้างและดูแล containers

เข้าใจเครื่องมือก่อนพิมพ์คำสั่ง

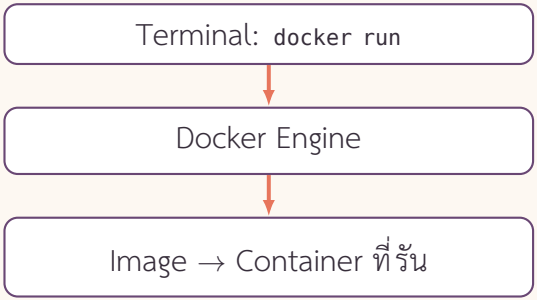
ร้านกาแฟ → Book API → Container → Docker Compose

Docker รับคำสั่งเพื่อจัดการ image และ container

Docker CLI รับคำสั่งจาก terminal แล้วส่งไปยัง Docker Engine ซึ่งจัดการ images และ containers

Image อาจดาวน์โหลดจาก registry เช่น Docker Hub. Engine ใช้ image สร้าง container ตามการตั้งค่าที่เราระบุ

รถบ้านช่วยนึกภาพชุดร้าน; Docker เป็นเครื่องมือที่ช่วยเตรียม เปิด ดู และหยุดชุดโปรแกรม



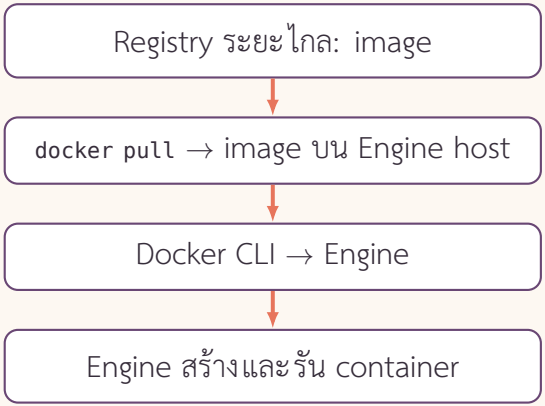
ลองคิดหรือทดลอง: ถ้า Engine ยังไม่เปิด คำสั่งจาก terminal จะสร้าง container ได้หรือไม่?

Registry เก็บ image ส่วน Engine จัดการบนเครื่องเรา

Registry เป็นที่เก็บ image ให้ทีมแบ่งปัน เช่น Docker Hub. ชื่อ `mongo:7.0` ระบุ image และ tag ที่ต้องการ

`docker pull` ให้ Engine ดาวน์โหลด image ไปเก็บบนเครื่องที่รัน Engine. Docker CLI ติดต่อ Engine ในเครื่องหรือ Engine ระยะไกลที่ตั้งค่าไว้

Engine ใช้ image ที่มีอยู่สร้างและจัดการ containers; การรัน container ไม่ได้หมายความว่ามันอยู่บน registry



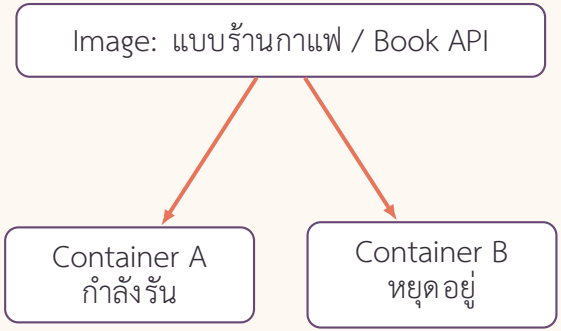
ลองคิดหรือทดลอง: เมื่อดาวน์โหลด image แล้ว container กำลังรันใน registry หรือบน Engine ที่เราเลือก?

Image เป็นชุดตั้งต้น ส่วน container เป็น instance

Image คล้ายแบบรถบ้านพร้อมรายการอุปกรณ์. Container คล้ายรถที่สร้างจากแบบนั้น โดยมีสถานะของตนว่าเป็น **กำลังรัน** หรือ **หยุด**

Image เดิมใช้สร้าง container ได้หลายตัว. การแก้ไฟล์ใน container ไม่ได้แก้ image ตั้งต้นโดยอัตโนมัติ

ตรวจ image ด้วย `docker image ls`; ตรวจ containers ด้วย `docker ps -a`



ลองคิดหรือทดลอง: `docker image ls` กับ `docker ps -a` บอกสิ่งเดียวกันหรือไม่?

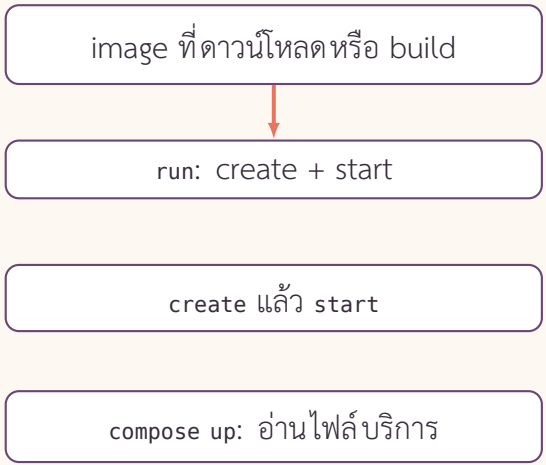
จาก image เราสร้าง container ได้หลายทาง

ทางที่ 1: `docker run` สร้าง container และเริ่มทำงานในคำสั่งเดียว

ทางที่ 2: `docker create` สร้าง container ที่ยังไม่รัน แล้ว `docker start` จึงเริ่ม container นั้น

ทางที่ 3: `docker compose up` อ่านไฟล์ Compose แล้วสร้างหรือใช้ containers ตามบริการ. ทั้งสามทางใช้ image เป็นจุดตั้งต้น

`docker build` สร้าง **image** จาก Dockerfile; ยังไม่ใช่การรัน container



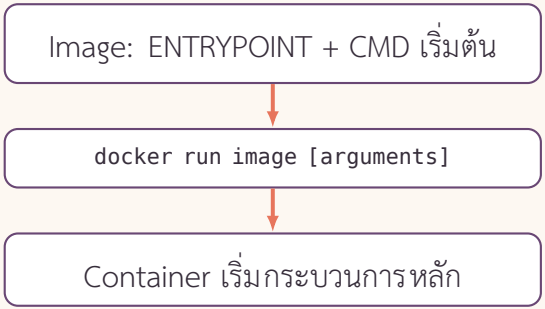
ลองคิดหรือทดลอง: ถ้าใช้ `docker build` อย่างเดียว มี container กำลังรันแล้วหรือยัง?

คำสั่งแรกของ container มาจาก image หรือผู้ใช้

Image ระบุคำสั่งเริ่มต้นไว้ได้ด้วย `ENTRYPOINT` และ `CMD` เมื่อใช้ `docker run image` Docker ใช้ค่าที่ image ระบุ

ผู้ใช้เพิ่มคำสั่งหรือ arguments หลังชื่อ image ได้ หาก image มี ENTRYPOINT อยู่ ค่าเหล่านี้มักกลายเป็น arguments ของ ENTRYPOINT แทน CMD เดิม

Compose กำหนด `command:` เพื่อเปลี่ยน CMD ได้ ตรวจ image และผลจริงก่อนสรุปว่าคำสั่งใดจะรัน



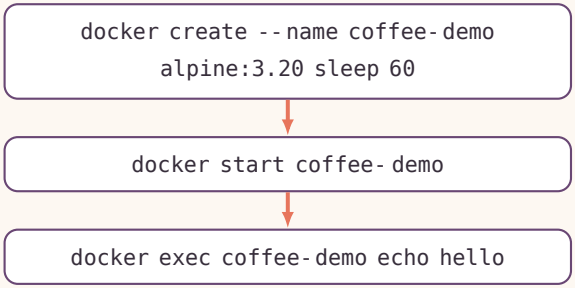
ลองคิดหรือทดลอง: ถ้า image มี ENTRYPOINT ค่า arguments หลังชื่อ image ส่งให้ส่วนใด?

คำสั่งที่ใช้จริงแสดงความต่างระหว่าง create และ exec

ตัวอย่างนี้ใช้ image `alpine:3.20` คำสั่ง `create` เตรียม container ชื่อ `coffee-demo` ให้รัน `sleep 60` แต่ยังไม่เริ่ม

`start` เริ่ม container เดิม ขณะที่มันยังรัน `exec` สั่ง `echo hello` เพิ่มภายใน container เดียวกัน

เมื่อ `sleep` จบ container จะหยุด ลบตัวทดลองด้วย `docker rm coffee-demo`



ลองคิดหรือทดลอง: หลังใช้ exec มี container ใหม่เพิ่มขึ้นหรือไม่? ตรวจด้วย `docker ps -a`

Exec เพิ่มคำสั่งใน container ที่กำลังรัน

`docker run` สร้าง container ใหม่แล้วเริ่มกระบวนการหลัก.
`docker start` เปิด container ที่สร้างไว้แล้วด้วยการตั้งค่าเดิม

`docker exec` เรียกคำสั่งเพิ่มใน container ที่กำลังรัน เช่น `docker exec -it dti201-web sh` เพื่อเข้า shell หาก image มี shell

เมื่อออกจาก shell ไม่ได้หมายความว่าเราได้สร้าง container ใหม่. ถ้า container หยุดอยู่ ต้องเริ่มมันก่อนจึงใช้ exec ได้

`run`: สร้าง + เริ่ม ตัว ใหม่

`start`: เริ่ม ตัว เดิม ที่ หยุด

`exec`: คำ สั่ง เพิ่ม ใน ตัว ที่ รัน

ลองคิดหรือทดลอง: ถ้าต้องการดูไฟล์ใน container ที่กำลังรัน ควรสร้างตัวใหม่หรือใช้ exec?

Compose ก็เปิดคำสั่งใน container ได้สองแบบ

`docker compose run SERVICE COMMAND` สร้าง container ชั่วคราวสำหรับคำสั่งนั้นตามการตั้งค่าบริการ; เติม `--rm` เมื่อต้องการลบ container หลังจบ

`docker compose exec SERVICE COMMAND` ใช้ container ของบริการที่กำลังรันอยู่. เช่น `docker compose exec db mongosh` เปิด shell ของ MongoDB ในบริการ `db`

`docker compose up` เริ่มบริการด้วยคำสั่งที่กำหนดใน image หรือ Compose; แต่ละวิธีมีจุดประสงค์ต่างกัน

`compose up`
เปิด บริการ ตาม แผน

`compose run`
สร้าง ตัว ใหม่ สำหรับ คำ สั่ง

`compose exec`
สั่ง เพิ่ม ใน ตัว ที่ กำลัง รัน

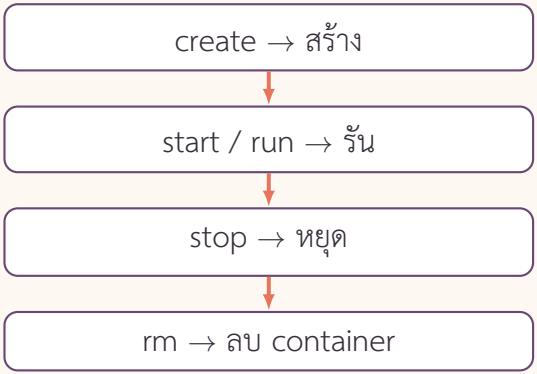
ลองคิดหรือทดลอง: ถ้า MongoDB บริการ db กำลังทำงานและต้องการเปิด mongosh ควรใช้วิธีใด?

คำสั่ง Docker ติดตามวงจรชีวิตของ container

`docker ps -a` ดูสถานะ; `docker logs` ดูข้อความจากกระบวนการหลัก

`stop` ขอให้หยุด; `start` เปิดตัวเดิม; `rm` ลบ container ที่หยุดแล้ว. Image ตั้งต้นยังคงอยู่

`docker run --rm` ให้ Docker ลบ container นั้นหลังคำสั่งจบ; volume ที่ระบุแยกไว้ไม่ได้ถูกลบเพียงเพราะ container จบ



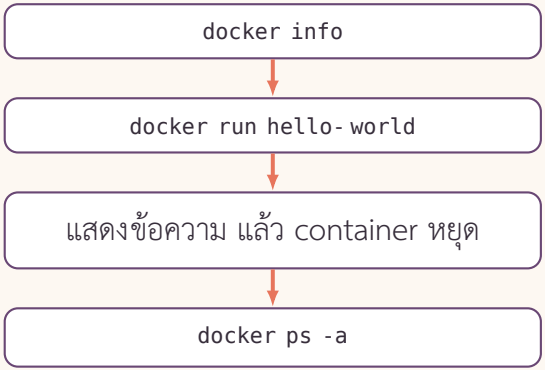
ลองคิดหรือทดลอง: container หยุดแล้ว เราใช้คำสั่งใดเปิดตัวเดิมโดยไม่สร้างตัวใหม่?

คำสั่งแรกสร้าง container แล้วให้มันทำงาน

`docker run hello-world` ให้ Docker หา image, สร้าง container และรันโปรแกรมในนั้น

โปรแกรมนี้แสดงข้อความแล้วจบ. `docker ps` แสดงตัวที่กำลังรัน ส่วน `docker ps -a` แสดงตัวที่หยุดแล้วด้วย

เริ่มด้วย `docker version` และ `docker info` เพื่อตรวจว่าเครื่องติดต่อ Engine ได้



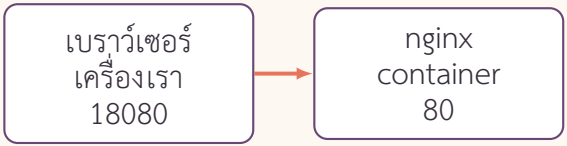
ลองคิดหรือทดลอง: ทำไม `docker ps` อาจไม่เห็น container ที่ทำงานเสร็จแล้ว?

เปิดเว็บตัวอย่างก่อนกลับไป Book API

ในใบงาน เราเปิด nginx ซึ่งเป็นเว็บเซิร์ฟเวอร์ใน container. เครื่องของเราเข้า port `18080`; ภายใน container nginx ฟัง port `80`

เปิดหน้าเว็บ ตรวจด้วย `docker ps` แล้วลองหยุดและเปิด container เดิม

ขั้นนี้ทำให้เราเห็น port และวงจรชีวิต ก่อนนำแนวคิดเดียวกันไปอ่าน MongoDB ของ Lab 10




```
-p 127.0.0.1:18080:80
```

ลองคิดหรือทดลอง: ถ้าเว็บไม่ขึ้น จะตรวจ Engine, container และ port ตามลำดับอย่างไร?

ช่วงที่ 4 / 4

Compose ช่วยเขียนแผนเปิดบริการหลายส่วน

จากขบวนรถร้านกาแฟสู่ไฟล์สำหรับ Book API และ MongoDB

ร้านกาแฟ → Book API → Container → Docker Compose

แผนขบวนรถบอกว่าแต่ละคันต้องเตรียมอะไร

สมมติร้านมีรถขายกาแฟหนึ่งคัน และรถเก็บวัตถุดิบอีกหนึ่งคัน. ก่อนเปิดงาน ทีมจดว่ามีรถอะไร แต่ละคันรับผิดชอบอะไร และต้องเชื่อมบริการใด

Compose ทำหน้าที่คล้ายแผนที่เขียนไว้: ระบุ services, images, ports และ volumes ให้ Docker อ่าน

Compose ไม่ได้หมายความว่าฐานข้อมูลพร้อมรับคำขอทันทีที่ container เริ่ม. เราต้องตรวจความพร้อมด้วย



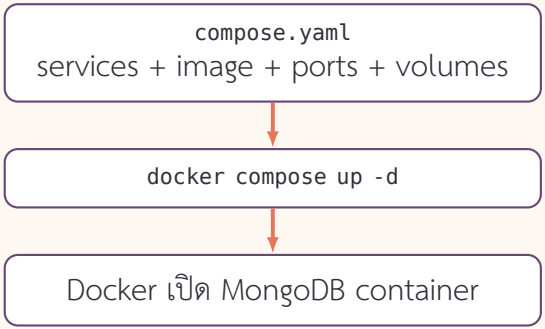
ลองคิดหรือทดลอง: ถ้ารถเก็บวัตถุดิบยังไม่พร้อม รถขายกาแฟควรตรวจอะไร?

ไฟล์ Compose ระบุสิ่งที่ต้องการให้ Docker เปิด

ไฟล์ `compose.yaml` เก็บรายละเอียดของบริการ. ใน Lab 10 บริการชื่อ `db` ใช้ image `mongo:7.0`

`ports` เชื่อม port ของเครื่องกับ port ใน container. `volumes` บอกพื้นที่ที่เก็บข้อมูล MongoDB

คำสั่ง `docker compose up -d` อ่านไฟล์แล้วสร้างหรือเปิดบริการที่กำหนด



ลองคิดหรือทดลอง: ไฟล์แผนอยู่ที่ใด และคำสั่งใดใช้แผนนั้นเปิดบริการ?

ไฟล์ MongoDB บอก image พอร์ตและ volume ทีละบรรทัด

ใน Lab 10 `db` เป็นชื่อบริการ. `image` ระบุ MongoDB รุ่น 7.0

`ports` เชื่อม `27018` บนเครื่องไป `27017` ใน container; `volumes` เชื่อมชื่อ `book_mongodata` ไปยัง `/data/db`

รายการ `volumes` ท้ายไฟล์ประกาศ named volume ให้ Compose จัดการ. เปิดไฟล์จริงของ Lab 10 เทียบทีละบรรทัด

```
services:
  db:
    image: mongo:7.0
    ports:
      - "127.0.0.1:27018:27017"
    volumes:
      - book_mongodata:/data/db
volumes:
  book_mongodata:
```

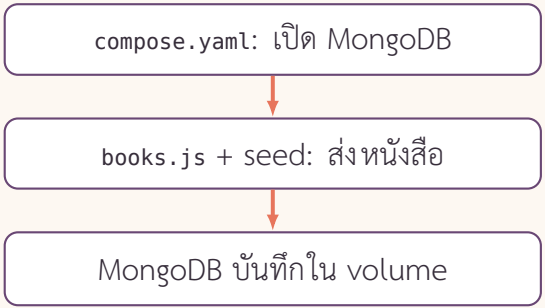
ลองคิดหรือทดลอง: ถ้า Express อยู่บน host ให้ใช้ port ใดติดต่อ MongoDB?

แผน Compose ไม่ได้บรรจุข้อมูลหนังสือ

Compose ระบุว่าจะเปิด MongoDB อย่างไร; ส่วนข้อมูลหนังสือ 200 เล่มมาจากไฟล์ `books.js` และ seed script ของ Lab 10

เมื่อ seed script เขียน documents ลง MongoDB ข้อมูลจึงอยู่ในฐานข้อมูลและ volume

แยกสามสิ่งนี้ให้ชัด: ไฟล์แผนบริการ, โปรแกรม seed และข้อมูลที่บันทึกแล้ว



ลองคิดหรือทดลอง: เปิด MongoDB สำเร็จแล้ว เหตุใดยังอาจไม่เห็นหนังสือ 200 เล่ม?

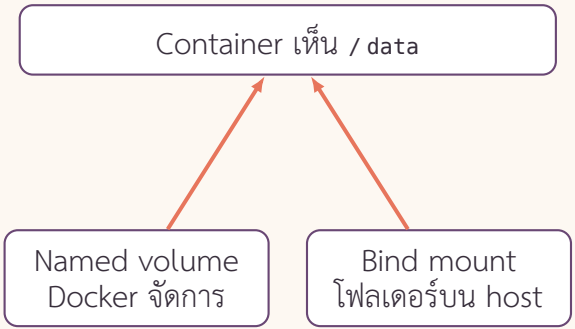
Volume และ bind mount เก็บข้อมูลไว้นอก container

ทั้งสองวิธีเชื่อมพื้นที่นอกชั้นเขียนของ container เข้ามายังตำแหน่งใน container

Named volume: Docker จัดการพื้นที่ เช่น `book_mongodata:/data/db` สำหรับไฟล์ MongoDB

Bind mount: เลือกโฟลเดอร์จริงบนเครื่องที่รัน Docker Engine เช่น `/path/books:/data`. ทั้งสองฝั่งเห็นไฟล์เดียวกัน; หาก Engine อยู่ระยะไกล โฟลเดอร์ต้องอยู่บนเครื่องนั้น

การลบ container ไม่ได้ลบโฟลเดอร์ host หรือ named volume โดยอัตโนมัติ แต่การลบไฟล์ในพื้นที่ที่ mount มีผลกับข้อมูลนั้นจริง



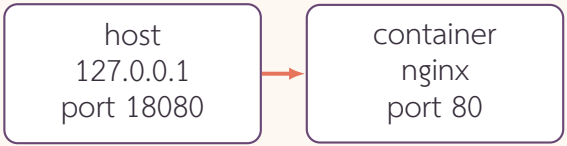
ลองคิดหรือทดลอง: ถ้าต้องแก้ source ในโฟลเดอร์เครื่องแล้วให้ container เห็นทันที วิธีใดเหมาะ?

Port mapping ระบุช่องทางจาก host ไป container

ใน `-p 127.0.0.1:18080:80` ค่าแรกเป็น address ที่รับบน host, 18080 เป็น port บน host และ 80 เป็น port ใน container

สำหรับ MongoDB ของ Lab 10 ให้ Express บน host ต่อ `127.0.0.1:27018`; Docker ส่งต่อไป port 27017 ของ MongoDB container

ถ้า host port ถูกใช้แล้ว ให้เลือก port ฝั่ง host ที่ว่าง และแก้ URL ของ client ให้ตรง



host address : host
port : container port

ลองคิดหรือทดลอง: เบราว์เซอร์บน host ต้องเปิด 18080 หรือ 80?

หยุดบริการกับลบข้อมูลเป็นคนละการกระทำ

`docker compose stop` หยุด containers ที่สร้างไว้. `docker compose start` เปิด containers เดิมอีกครั้ง

`docker compose down` หยุดและลบ containers กับ network ของ Compose; named volumes ยังอยู่โดยค่าเริ่มต้น

`docker compose down -v` ลบ named volumes ที่ประกาศในไฟล์ด้วย. ใช้เฉพาะเมื่อแน่ใจว่าจะลบข้อมูลทดลองนั้น

`stop` → หยุด

`down` → ลบ container

`down -v` → ลบ volume ด้วย

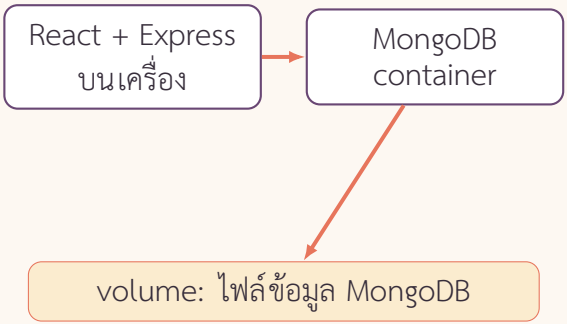
ลองคิดหรือทดลอง: ถ้าต้องเก็บข้อมูลหนังสือไว้ คำสั่งใดต้องระวังเป็นพิเศษ?

Book API กับ MongoDB ต่อกันเป็นระบบเดียว

Lab 09 สร้าง Book API ด้วย Express บน Node.js. Lab 10 เพิ่ม MongoDB เพื่อให้หนังสือยังอยู่เมื่อ Express หยุดและเปิดใหม่

ในใบงาน Docker เราเริ่มจาก nginx container และทดลอง volume ก่อน แล้วจึงอ่าน Compose ของ MongoDB

เมื่อจบ ให้ตามหนังสือเล่มเดียว: React ส่งคำขอ → Express → MongoDB → volume แล้วดูคำตอบกลับ



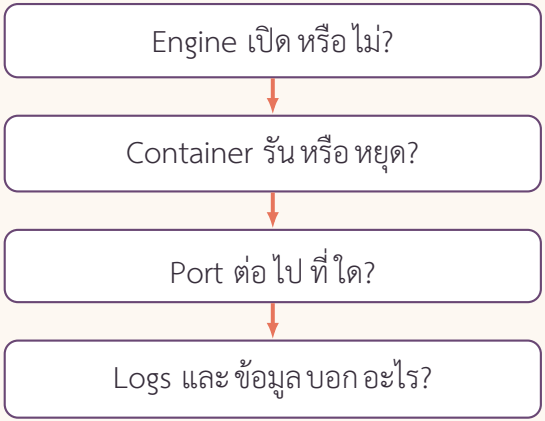
ลองคิดหรือทดลอง: ชี้ส่วนที่เป็นโปรแกรม ส่วนที่เป็น container และส่วนที่เก็บข้อมูล

เราตรวจแต่ละชั้นได้เมื่อสิ่งใดไม่ทำงาน

ถ้า Docker ติดต่อ Engine ไม่ได้ ให้ตรวจว่า Engine เปิดอยู่

ถ้า container ไม่รัน ให้ดู `docker ps -a` และ `docker logs`. ถ้าเว็บไม่ตอบ ให้ตรวจ port บนเครื่องและ port ใน container

ถ้า MongoDB เปิดแล้วแต่ยังไม่มีหนังสือ ให้ตรวจว่า seed ทำงานแล้วหรือยัง และกำลังดูฐานข้อมูลที่ถูกตัวหรือไม่



ลองคิดหรือทดลอง: ถ้า port ตอบไม่ได้ ต้องดูอะไรเป็นหลักฐานก่อนแก้?

ภาพร้านช่วยจำ แต่ระบบจริงต้องอธิบายด้วยหลักฐาน

บ้าน เต็นท์ และรถบ้านช่วยให้เห็นการเตรียมสถานที่และอุปกรณ์. คอมพิวเตอร์จริงมี image, process, kernel, port และ volume ที่ต้องแยกให้ออก

ในแล็บ ให้แสดงเว็บใน container, อ่านไฟล์จาก volume โดยใช้ container อีกตัว และอธิบายไฟล์ Compose ที่เขียนเอง

สุดท้ายวาด stack: Book API → Node.js → kernel → processor แล้วชี้ว่า container เพิ่มการแยกตรงไหน



ลองคิดหรือทดลอง: อธิบายด้วยคำของตนเอง: image, container และ volume ทำหน้าที่ต่างกันอย่างไร?

คำอธิบายที่ตรวจได้อยู่ในเอกสารทางการ

Docker Docs: What is a container?

Docker Learn: containers and virtual machines

Docker Docs: Compose Quickstart

Docker Docs: Compose down and volumes

ใช้ใบงาน Docker แยกควบคู่กับบทเรียน แล้วกลับไป Lab 10